Name	Link	Comment

You can absolutely create a brand-new AWS Amplify Gen 2 application from scratch using Kiro; the application does not need to be already created. Kiro features an integrated **AWS Amplify Power** that dynamically activates to guide you through the entire lifecycle—from initialization to deployment. [1, 2, 3]

If you are starting fresh, Kiro can help you scaffolding your project, setting up the file directory, and provisioning the backend. [1, 2]

How to Create a Gen 2 App with Kiro

Initialize the local project: You (or Kiro, if given terminal permissions) can start a fresh frontend app (e.g., using Vite and React) and initialize Amplify Gen 2 directly in your root directory by running: [1, 2]

bash

```
npm create vite@latest my-amplify-app -- --template react-ts
cd my-amplify-app
npm create amplify@latest
```

1. Usa el código con precaución.

Configure your AWS credentials: Ensure your local machine has a configured profile so Kiro and the underlying tools can communicate with your AWS account: [1, 2]

bash

```
aws configure
```

2. Usa el código con precaución.
3. **Build out the backend:** You can ask Kiro to write or modify code inside the newly generated `amplify/` folder. Kiro can design your TypeScript-based data models (`amplify/data/resource.ts`), authentication configuration (`amplify/auth/resource.ts`), and serverless functions. [1, 2, 3]

Deploy a local sandbox: Kiro can guide you or use hooks to trigger the real-time local cloud environment so you can test your infrastructure instantly: [1]

bash

```
npx ampx sandbox
```

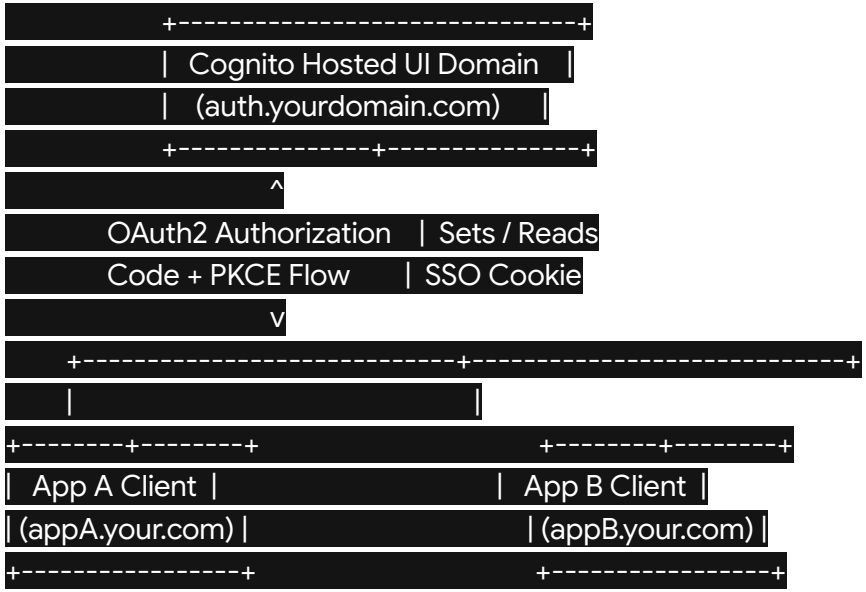
4. Usa el código con precaución.

Whether you want to build a completely new expense tracker, a To-Do application, or an AI-driven chat app, you can prompt Kiro right now to get started. [1, 2, 3]

Would you like me to generate the **TypeScript configuration schema** for your new app's backend, or should we start by selecting your **frontend framework** (like React, Next.js, or Vue)?

To share an Amazon Cognito User Pool across multiple web applications using AWS Amplify, you rely on **OAuth 2.0 PKCE redirection via Cognito Hosted UI / Managed Login**.
When all client apps share a central domain (e.g., auth.yourdomain.com), Cognito writes a session cookie to that domain. Once a user logs into App A, navigating to App B triggers an OAuth redirect to Cognito; Cognito detects the active session cookie and immediately redirects back to App B with an authorization code without requiring re-entry of credentials.

Core Architecture



Step-by-Step Implementation

1.Configure the Shared Cognito User Pool Domain: Prerequisite for central SSO token cookies.

1. Open the Amazon Cognito Console and navigate to your User Pool.
2. Under **App integration**, edit the **Domain name**.

3. Set up a custom domain (e.g., auth.yourdomain.com) or use a Cognito prefix domain (your-app-prefix.auth.us-east-1.amazoncognito.com).

2.Register App Clients & Redirect URLs: Define origins and callback endpoints.
You can either use a **single App Client** configured with multiple callback URLs or **separate App Clients** for each app (recommended for granular permissions and isolated client secrets).

In the Cognito App Client settings, configure the following:

- **Allowed Callback URLs:**
  - https://appA.yourdomain.com/
  - https://appB.yourdomain.com/
- **Allowed Sign-out URLs:**
  - https://appA.yourdomain.com/logout
  - https://appB.yourdomain.com/logout
- **Allowed OAuth Flows:** Authorization Code Grant (PKCE enabled)
- **Allowed OAuth Scopes:** openid, email, profile

3.Configure AWS Amplify on App A and App B: Configure Amplify Auth with Hosted UI redirection.
In both application codebases, configure Amplify.configure() using the exact same User Pool ID, User Pool Web Client ID, and Cognito Domain.

```
TypeScript
import { Amplify } from 'aws-amplify';

Amplify.configure({
  Auth: {
    Cognito: {
      userPoolId: 'us-east-1_XXXXXXX',
      userPoolClientId: 'YOUR_APP_CLIENT_ID', // Can be shared or app-specific
      loginWith: {
        oauth: {
          domain: 'auth.yourdomain.com', // Or Cognito domain prefix
          scopes: ['openid', 'email', 'profile'],
          redirectSignIn: [
            // Set the specific URL matching the local running app
            window.location.origin + '/'
          ],
          redirectSignOut: [
            window.location.origin + '/logout'
          ],
          responseType: 'code' // Requires Authorization Code Grant for PKCE
        }
      }
    }
  }
});
```

4.Trigger Federated Sign-In in Frontend Code: Use signInWithRedirect to handle SSO.

Instead of rendering custom local username/password forms in each app, trigger signInWithRedirect via @aws-amplify/auth.

```
TypeScript
import { signInWithRedirect, signOut, getCurrentUser } from 'aws-amplify/auth';

// 1. Trigger redirection to Cognito Hosted UI
const handleLogin = () => {
  signInWithRedirect();
};

// 2. Check current authentication state on app load
const checkUser = async () => {
  try {
    const user = await getCurrentUser();
    console.log('User signed in:', user);
  } catch (err) {
    // Not signed in locally -> Redirecting to Hosted UI will automatically
    // sign them in if a valid SSO cookie exists at auth.yourdomain.com
    handleLogin();
  }
};

// 3. Global SSO Logout
const handleLogout = () => {
  signOut({ global: true }); // Clears session across Cognito domain
```

Important Considerations

Key Aspect	Recommendation
Domain Strategy	Use a custom domain (e.g., auth.yourdomain.com) sharing the same parent domain as your web applications (e.g., appA.yourdomain.com, appB.yourdomain.com).
Token Storage	Amplify handles tokens in memory and local storage per web origin. Silent authentication relying on the central SSO cookie runs during the OAuth redirect loop.
Global Sign-Out	Ensure signOut({ global: true }) is invoked when logging out so Cognito revokes refresh tokens and clears the SSO domain session.

In AWS Amplify Gen 2, backend resources are defined strictly in TypeScript using @aws-amplify/backend.
To configure a shared Cognito User Pool that handles SSO across multiple applications (app1.yourdomain.com, app2.yourdomain.com), you combine defineAuth for OAuth settings with **AWS CDK escape hatches** in amplify/backend.ts to attach a Cognito Domain.

Step 1: Define Auth in Primary Application (Central Auth Owner)

In your main backend project, set up amplify/auth/resource.ts using defineAuth. Add all allowed callback and logout URLs for all client applications.

```TypeScript
// amplify/auth/resource.ts
import { defineAuth } from '@aws-amplify/backend'

export const auth = defineAuth({
  loginWith: {
    email: true
    // Configure OAuth / Hosted UI redirect endpoints
    externalProviders: {
      callbackUrls: [
        'http://localhost:3000/api/auth/callback'
        'https://app1.yourdomain.com/'
        'https://app2.yourdomain.com/'
      ]
      logoutUrls: [
        'http://localhost:3000/'
        'https://app1.yourdomain.com/logout'
        'https://app2.yourdomain.com/logout'
      ]
    }
  }
})
```

Step 2: Attach the Cognito Domain via CDK Escape Hatch

Cognito SSO requires a shared domain (e.g., auth.yourdomain.com) to host the central auth session cookie. Use the CDK escape

hatch in amplify/backend.ts to associate a prefix or custom domain with the User Pool.

```
TypeScript
// amplify/backend.ts
import { defineBackend } from '@aws-amplify/backend'
import { auth } from './auth/resource'

const backend = defineBackend({
  auth
})

// Access underlying CfnUserPool & attach a domain prefix
const userPool = backend.auth.resources.userPool

userPool.addDomain('CognitoSSODomain', {
  cognitoDomain: {
    domainPrefix: 'my-shared-sso-app' // Creates my-shared-sso-app.auth.<region>.amazoncognito.com
  }
})
```

Step 3: Consume Shared Resources in Secondary Apps

For secondary applications (App B, App C), reference the existing centralized User Pool instead of creating a new backend.

Option A: Reference in Codebase via amplify/auth/resource.ts

Use referenceAuth to bind Amplify Gen 2 directly to the primary environment:

```TypeScript
// amplify/auth/resource.ts (App B)
import { referenceAuth } from '@aws-amplify/backend'

export const auth = referenceAuth({
  userPoolId: 'us-east-1_XXXXXXX',
  identityPoolId: 'us-east-1:xxxx-xxxx-xxxx',
  authRoleArn: 'arn:aws:iam::123456789012:role/amplify-authenticated-role',
  unauthRoleArn: 'arn:aws:iam::123456789012:role/amplify-unauthenticated-role',
  userPoolClientId: '1234567890abcdef',
});
```

Option B: Configure Frontend Client Directly (amplify_outputs.json / Amplify.configure)

Alternatively, pass the shared parameters directly when initializing the frontend:

```TypeScript
// main.tsx / app.tsx (App B)
import { Amplify } from 'aws-amplify'

Amplify.configure({
  Auth: {
    Cognito: {
      userPoolId: 'us-east-1_XXXXXXX',
      userPoolClientId: 'YOUR_APP_CLIENT_ID',
      loginWith: {
        oauth: {
          domain: 'my-shared-sso-app.auth.us-east-1.amazoncognito.com',
          scopes: ['openid', 'email', 'profile'],
          redirectSignIn: ['https://app2.yourdomain.com/'],
```

```
        redirectSignOut: 'https://app2.yourdomain.com/logout',
        responseType: 'code'
    }
  }
}
```